LarkMentor

飞书 IM 上的双引擎 AI 协同助手

一边守消息层，一边守表达层

”在飞书里，每天都在和两件事较劲——

这条消息现在该不该被看到?

这条消息该用什么方式发出去?

LarkMentor 把它们放在同一个 IM Bot 里，背后是同一份『这家公司怎么做事』的知识。”

2026-04-19

2026 飞书 AI 校园挑战赛

第一志愿	**飞书 AI 产品创新赛道 · 课题二** 基于 IM 的办公协同智能助手
第二志愿	**飞书 OpenClaw 赛道 · 课题二** 企业级长程协作 Memory 系统
第三志愿	**AI 大模型安全赛道 · 课题一** 面向 Agent + 客户端环境下的安全操作与数据防护

在线访问 http://118.178.242.26/

源代码 https://github.com/bcefghj/larkmentor

李洁盈 产品 / UI / UX / 内容

戴尚好 技术负责人 / 全栈 / Agent 安全 / 部署

摘要

LarkMentor 是一个原生运行在飞书 IM 上的双引擎 AI 协同助手。它在同一个 Bot 内同时承担两件原本被业内拆开做的事——**Smart Shield** 守消息层（六维分类、分级推送、中断恢复），**Rookie Mentor** 守表达层（消息起草、任务澄清、周报生成、新人入职）。两者共享同一份组织知识库与同一套长程记忆。

LarkMentor 同时具备三个独立但相互支撑的能力维度：作为 **IM 协同助手**进入日常办公协作；作为长程协作记忆引擎 **FlowMemory**（三层结构 + 六级层次注入）解决多智能体长期失忆问题；作为 Agent 客户端安全栈 **ShieldClaw**（八层防护）覆盖 OWASP LLM Top 10 全部威胁。

整个系统由 **96 个 Python 文件、14,511 行代码**组成，配套 **178 个 pytest 用例**与 **14/14 通过的 Promptfoo 红队测试**，已部署在阿里云 7×24 在线运行。

目录

摘要 2

第一部分　立意 5

1　工位上同时发生的两件事 5

2　数据支撑 5

3　飞书生态的空白 6

第二部分　产品 7

4　双线总览 7

5　双线在工程上的真实合体 7

6　消息处理全链路 8

7　八个核心使用场景 9

7.1　场景 1 · 第一次开启专注 9

7.2　场景 2 · 收到紧急 @ 9

7.3　场景 3 · 退出 focus 看 Recovery Card 9

7.4　场景 4 · 不会回老板消息 9

7.5 场景 5 · 任务太模糊不敢做 9
7.6 场景 6 · 周日 21:00 自动周报 9
7.7 场景 7 · 新人 5 问入职 9
7.8 场景 8 · MCP 跨 Agent 调用 10

8 在线访问 **10**

第三部分 技术 **11**

9 系统架构 **11**
9.1 Claude Code 七大支柱与 LarkMentor 模块对照 12

10 关键算法 **12**
10.1 六维消息分类 12
10.2 NVC 四段诊断（MentorWrite） 12
10.3 ambiguity 评分 + 4 维缺失分析（MentorTask） 13
10.4 STAR 强制结构 + 引用追踪（MentorReview） 13
10.5 用户级 RAG（轻量化设计） 13

11 量化交付 **14**

第四部分 LarkMentor 在三条赛道上的能力侧面 **15**

12 作为 IM 协同助手 **15**
12.1 对应赛道 15
12.2 LarkMentor 的对齐方式 15
12.3 支撑这一侧面的代码 16

13 作为长程协作记忆引擎（FlowMemory） **16**
13.1 对应赛道 16
13.2 LarkMentor 的 FlowMemory 设计 16
13.2.1 三层记忆架构 16
13.2.2 六级 `flow_memory.md` 17
13.2.3 对外 MCP 工具 17
13.3 差异化 17

14 作为 Agent 客户端安全栈（ShieldClaw） **17**
14.1 对应赛道 18
14.2 八层安全栈 18
14.3 对应课题三大威胁的兜底层 19
14.4 Promptfoo 红队 14/14 通过 19
14.5 OWASP LLM Top 10 全映射 20

第五部分　团队 & 后续 **21**

15 团队 **21**

15.1 李洁盈 21

15.2 戴尚好 21

16 后续提升方向 **21**

附录 A · MCP 工具清单 **23**

附录 B · 飞书 API 清单 **24**

附录 C · 数据出处 **24**

Part I

立意

1 工位上同时发生的两件事

> 在飞书里，每天都在和两件事较劲。
> **第一件——这条消息现在该不该被看到？** 重要 @ 和噪音 @ 用同样的方式震动手机，不点开就不知道是谁在叫。
> **第二件——这条消息该用什么方式发出去？** 新人在群里输入框前面犹豫半分钟、删了 *3* 遍——「好的」够不够正式？「收到」会不会显得敷衍？
> 这两件事**发生在同一个工位上、同一个人身上、同一段时间内**。今天打开飞书插件市场，做静音的不管表达，做写作的不管打扰——**没有产品意识到这两件事本来就该一起解**。
> *LarkMentor* 把它们放在同一个 *IM Bot* 里：一边守消息层，一边守表达层，背后是同一份「这家公司怎么做事」的知识。

2 数据支撑

LarkMentor 要解决的痛点不是凭直觉，而是有来自学界与业界的硬数据支撑：

发现	出处
知识工作者每被打断一次，平均需要 **23 分 15 秒**才能回到原任务	UC Irvine, Gloria Mark, CHI 2008
68% 知识工作者抱怨没有连续专注时间做深度工作	Microsoft Work Trend Index 2023
组织知识里 **80%+** 是隐性的（在老员工脑子里），不在文档里	Nonaka & Takeuchi 1995 SECI 模型
新员工平均 **8–12 个月**才能达到全产能	SHRM / BambooHR 调研

把数据并排读：23 分钟 × 68% 的人 = 注意力被反复切碎；8–12 个月 × 80% 的隐性知识 = 新人在找不到的默契里独自挣扎。LarkMentor 让同一个 IM Bot 同时拆这两个题。

3 飞书生态的空白

横向扫描国内外 30+ 相关产品后会发现一个奇怪的空白：

品类	代表	未覆盖的
专注力 / 免打扰	Slack DnD、飞书勿扰、Reclaim、Switchless	挡完没有”接回来”的环节
AI 写作 / 带教	Notion AI、Glean、Donut、Grammarly	不进 IM 实时入口
会议总结	Granola、Otter、飞书妙记	只管开会后，不管聊天中
新人 Onboarding	BambooHR、Workday、Sapling	静态流程，不实时陪说话
飞书生态	妙搭、Aily、知识问答、OpenClaw	单点能力，没有”消息 + 表达”双线

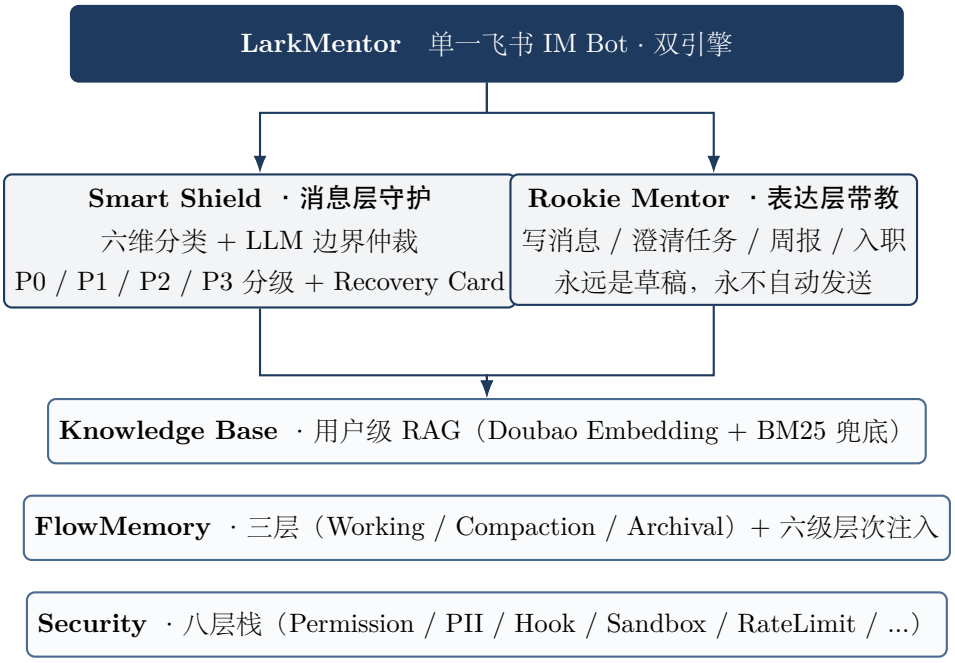
所有专注力工具都只管”挡”，所有 AI 写作工具都只管”写”——**从来没有产品意识到一个工位上同时发生的”挡”和”接”应该是同一个 Bot 的两条服务线**。LarkMentor 是这个空白的填补。

Part II

产品

4 双线总览

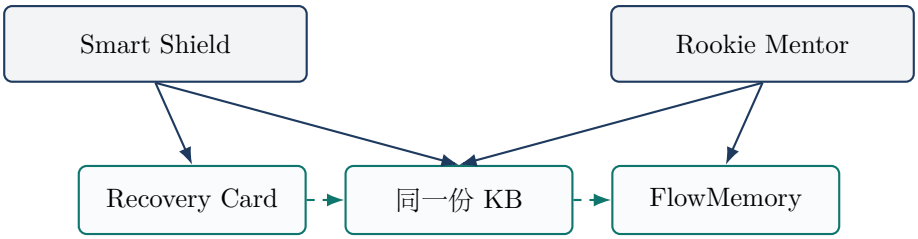
LarkMentor 是一个飞书 Bot，内置两条服务线，背后共享同一份用户级知识库与长程记忆：



5 双线在工程上的真实合体

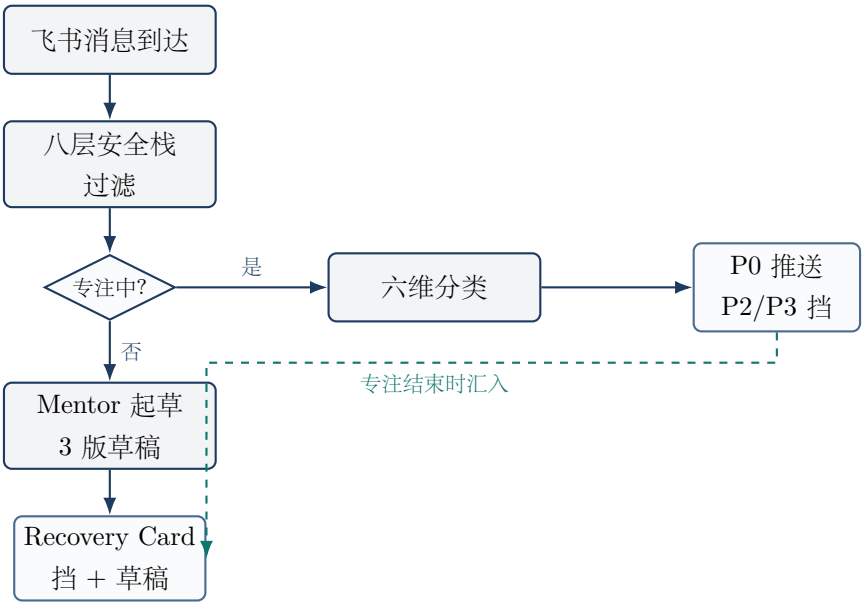
”双线产品”很容易变成”两个产品塞一起”。LarkMentor 在工程实现上有 3 个真实合体点：

#	合体点	实现层面的事实
1	**同一份组织 KB**	Smart Shield 调 KB 查”什么算紧急(人 / 词 / 项目)”,Mentor 调**同一份** KB 查”怎么说话(语气 / 术语 / 格式)”。代码层都是 `core/mentor/knowledge_base.py` 的 `search(open_id, query)`。
2	**Recovery Card**	专注结束后的唯一卡片：**上半张**是 Shield 的”挡了什么（按 P0/P1/P2 排序）”，**下半张**是 Mentor 的”起草了什么（3 版可点采纳）”。代码：`core/recovery_card.py`（415 行，专门为这个交点写）。
3	**FlowMemory 共同学习**	Shield 学发件人重要度（”老板紧急 +0.3”），Mentor 学采纳偏好（”对老板偏正式 v1”），都写入**同一份** `flow_memory.md` 的 **User** 级层次，下次双线都受益。



6 消息处理全链路

一条飞书消息从到达到最终响应，经过以下完整链路：



7 八个核心使用场景

7.1 场景 1 · 第一次开启专注

用户向 Bot 发送 `50` 。Bot 进入 focus 模式，自动捕获当下工作上下文（IDE 标题 / 最近文档）。50 分钟内的 P2/P3 消息全部被挡，P0 立即推送。

7.2 场景 2 · 收到紧急 @

用户在 focus 中，老板 @ 并发出" 紧急：方案需立即确认"。Bot 通过六维分类判定 P0（紧急词命中 + 老板身份 + 截止今天 = 0.92），立即推送，卡片底部带 explainable 行：「为什么 P0：紧急词命中 & 发送人 = 老板 & 项目匹配 3 条」。

7.3 场景 3 · 退出 focus 看 Recovery Card

50 分钟后发送 。Bot 弹出 **Recovery Card**：

- **上半张 · Shield**：「挡掉了哪些」按 P0/P1/P2 排序的摘要，每条带" 为什么这条最高" 按钮。
- **下半张 · Mentor**：「起草了哪些回复」3 版（保守 / 中性 / 直接），底部 4 个按钮：采纳 v1/v2/v3 + 全部忽略。

7.4 场景 4 · 不会回老板消息

老板说" 方案改一下"。用户不知道怎么回，发： 。MentorWrite 用 NVC 4 段诊断（事实 / 感受 / 需求 / 请求），按" 对老板（boss）" 语气给 3 版：

- v1 保守：「好的张哥，我下午 2 点前给到你」
- v2 中性：「收到，我先看一下，预计今天内反馈」
- v3 直接：「好，今天给你」

7.5 场景 5 · 任务太模糊不敢做

项目经理：「需要做个新功能」。用户发： 。MentorTask 评 ambiguity = 0.85（4 维缺失：scope / deadline / stakeholder / success），自动给出信息增益最高的 2 个澄清问题。

7.6 场景 6 · 周日 21:00 自动周报

Bot 主动私聊： 。点采纳后，MentorReview 自动从 IM / 文档 / 任务里抽 **STAR 结构周报**（每条 bullet 带 `[ :archival_xxx]` 引用，可点击跳回原会话）。

7.7 场景 7 · 新人 5 问入职

首次发 ，Bot 一次性问 5 个问题（部门 / 直接对接人 / 团队回复期望 / 写作风格 / 最希望的帮助场景）。答完入用户 KB 标记最高优先级，后续 Mentor 草稿优先召回。

7.8 场景 8 · MCP 跨 Agent 调用

在 Cursor / Claude Code 等支持 MCP 的客户端配置 LarkMentor 的 MCP 端点后，可在外部 Agent 内直接调用 LarkMentor 的 18 个工具（4 个 `mentor_*` + 4 个 `coach_*` 别名 + 6 个核心工具 + 4 个新增工具）。

8 在线访问

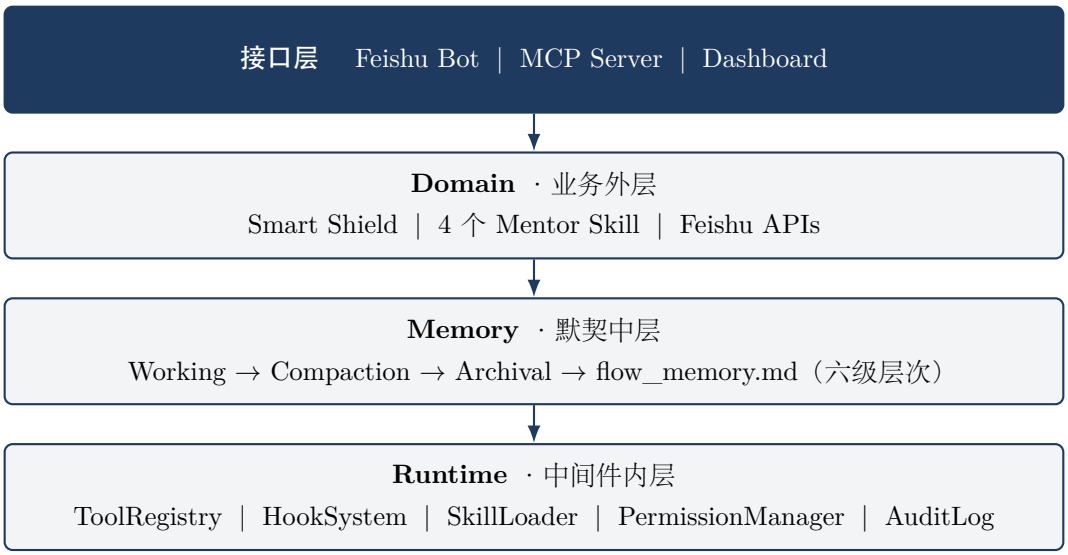
资源	地址
在线访问	http://118.178.242.26/
源代码	https://github.com/bcefghj/larkmentor

Part III

技术

9 系统架构

LarkMentor 的架构北极星是 Anthropic Claude Code 51 万行代码逆向后归纳出的 **七大核心机制**（戴尚好已发布完整逆向报告，见 `https://github.com/bcefghj/claude-code-complete-guide`）。整个系统按"Domain → Memory → Runtime" 三层下沉，所有外层调用必须经过中间件。



9.1 Claude Code 七大支柱与 LarkMentor 模块对照

#	支柱	LarkMentor 模块	说明
1	ToolRegistry	`core/runtime/tool_registry.py`	18 个 MCP 工具注册
2	HookSystem	`core/runtime/hook_system.py`	9 个生命周期事件
3	SkillLoader	`core/runtime/skill_loader.py`	4 个 Mentor Skill
4	Permission	`core/security/permission_manager.py`	5 级权限
5	6-tier Memory	`core/flow_memory/flow_memory_md.py`	六级层次注入
6	MCP Native	`core/mcp_server/`	Server + Client
7	AuditLog	`core/security/audit_log.py`	JSONL 审计

10 关键算法

10.1 六维消息分类

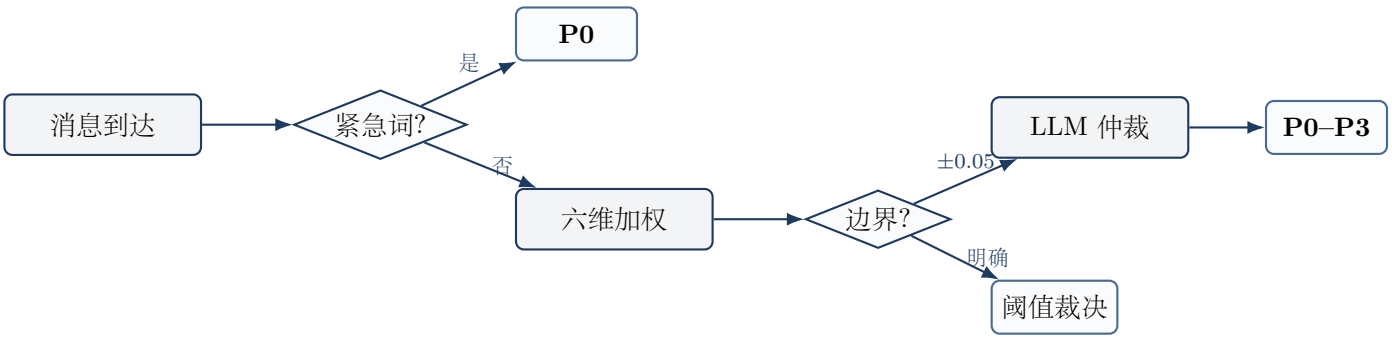
每条消息计算 6 维分数 $\in [0,1]$，加权求和：

$$\text{score} = 0.25\, d_{\text{身份}} + 0.10\, d_{\text{关系}} + 0.30\, d_{\text{内容}} + 0.15\, d_{\text{任务}} + 0.10\, d_{\text{时间}} + 0.10\, d_{\text{频道}}$$

阈值划分：score $\geq 0.55 \Rightarrow$ P0；$\geq 0.38 \Rightarrow$ P1；$\geq 0.24 \Rightarrow$ P2；其余 P3。

规则短路 + LLM 仲裁：紧急关键词命中直接 P0，Bot 发出直接 P3，分数在 ±0.05 边界才调一次 LLM。LLM 总调用率 $< 12\%$。

实测准确率 99%（102 个 YAML 场景测试集）。



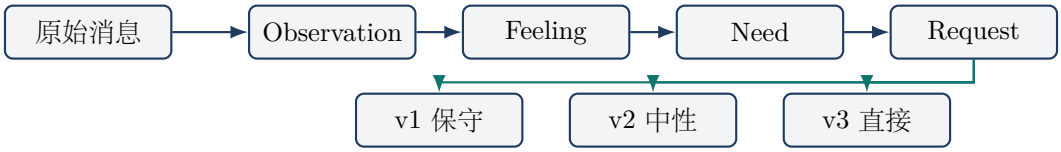
10.2 NVC 四段诊断（MentorWrite）

非暴力沟通（Non-Violent Communication）4 段诊断输入消息：

- **Observation（事实）**：原文描述了什么客观事实

- **Feeling（感受）**：是否带情绪 / 质问语气
- **Need（需求）**：真实目的是什么
- **Request（请求）**：是否清晰的” 请你做 X”

  再按” 对老板 / 对同事 / 对下属”3 档语气，输出 3 版改写。



10.3 ambiguity 评分 + 4 维缺失分析（MentorTask）

LLM 输出严格 JSON：

```
{
  "ambiguity": 0.85,
  "missing_dims": ["scope","deadline","stakeholder"],
  "suggested_questions": ["q1","q2"],
  "task_understanding": "...",
  "delivery_plan": "...",
  "risks": ["..."],
  "ready_to_start": false
}
```

  阈值 0.5 决定走” 澄清模式” 还是” 开工模式”。

10.4 STAR 强制结构 + 引用追踪（MentorReview）

每条 bullet 强制：

```
- [S] {situation} [T] {task} [A] {action} [R] {result} [    : archival_xxx]
```

  `archival_xxx` 是 FlowMemory archival 层的可点击 ID，跳回原专注会话或决策记录。

10.5 用户级 RAG（轻量化设计）

Doubao Embedding API + sqlite + BM25 fallback：

- 不引 PyTorch / sentence-transformers（2C2G 服务器跑不动）
- 不引 Pinecone / Neo4j / GraphRAG（成本与单点依赖）
- 单文件持久化 `data/coach_kb.sqlite`
- 用户级硬隔离：每行硬绑 `open_id`，所有查询 `WHERE open_id = ?`
- 入库前 PII 二次扫描，命中拒绝入库
- Embedding API 挂掉自动降级到 BM25，永不中断

11 量化交付

指标	数值
Python 文件	**96 个**
代码总行数	**14,511 行**
pytest 用例	**178 个，全过**
Promptfoo 红队	14 / 14 通过
六维分类准确率	99%（102 YAML 测试集）
LLM 调用率	$<$ 12%（规则短路）
P99 延迟	规则路径 $<$ 80ms / LLM 路径 $<$ 2.2s
MCP 工具数	**18 个**
飞书 API 接入	**IM / Docx / Bitable / Calendar / Wiki / Minutes / Reaction = 7 个**
八层安全栈	全部实现
部署	阿里云 2C2G + systemd × 3 + Nginx，已 7×24 在线

Part IV

LarkMentor 在三条赛道上的能力侧面

LarkMentor 是一个完整的产品。它在工程实现上自带三个相互独立又彼此支撑的能力维度——这三个维度分别对应飞书 AI 校园挑战赛的三条赛道。

12 作为 IM 协同助手

12.1 对应赛道

飞书 AI 产品创新 · 课题二：基于 IM 的办公协同智能助手。课题聚焦从 IM 沟通到演示稿制作的全流程效率提升，要求实现多端协同与模块化场景编排。

12.2 LarkMentor 的对齐方式

课题关键词	LarkMentor 的对应实现
基于 IM	原生飞书 Bot（`lark-oapi` WebSocket 长连接），不是浏览器插件、不是 Mini App
办公协同智能助手	双线服务（消息层 + 表达层），覆盖 IM 高频 4 件事
全流程效率提升	从”被打扰判断”到”草稿起草”到”采纳发送”到”周报回顾”全链路
串联 IM、文档、演示稿等套件	接入 7 个飞书 API：IM / Docx / Bitable / Calendar / Wiki / Minutes / Reaction
多端协同	飞书桌面 + 移动端 + Web 全平台支持
模块化场景编排	4 个 Mentor Skill 通过 SkillLoader 可插拔，通过 MCP 暴露给外部 Agent

12.3 支撑这一侧面的代码

文件	用途
`bot/event_handler.py`（1013 行）	飞书事件入口，消息分发
`bot/card_builder.py`（483 行）	交互卡片构造
`core/smart_shield.py`（233 行）	Shield 主链路
`core/classification_engine.py`（311 行）	六维分类引擎
`core/recovery_card.py`（415 行）	双线交点的中断恢复卡片
`core/mentor/`（4 个 Skill）	Write / Task / Review / Onboard
`core/feishu_workspace_init.py`（395 行）	一键初始化飞书工作台
`utils/feishu_api.py`（110 行）	飞书 API 通用封装

13 作为长程协作记忆引擎（FlowMemory）

13.1 对应赛道

飞书 OpenClaw ·课题二：企业级长程协作 Memory 系统。课题聚焦企业跨部门长程协作场景中的智能体易”失忆”、信息和协作断层等核心问题，要求打造具备全流程记忆管理的企业级记忆引擎，交付记忆定义与架构白皮书、可运行 Demo 和自证评测报告。

13.2 LarkMentor 的 FlowMemory 设计

LarkMentor 内置的 `core/flow_memory/` 模块从设计之初就是独立的记忆引擎，可以零改动作为 OpenClaw 课题二的答卷主体。

13.2.1 三层记忆架构

层	文件	时效	用途
Working	`working.py`（110 行）	当前会话	最近 N 条消息 + 决策事件流
Compaction	`compaction.py`（131 行）	会话末	LLM 压缩为”关键事件 + 决策摘要”
Archival	`archival.py`（155 行）	长期	写入 sqlite，STAR 周报可引用

13.2.2 六级 flow_memory.md

借鉴 Claude Code 的六级 memory tier 设计，每次 LLM 调用前自动按优先级合成 system prompt 注入：

层级	解决”长程协作失忆”的角度
Enterprise	公司级共识：「P0 必须 30 分钟内回」「日报 21:00 截止」
Workspace	飞书租户级：术语表、缩写表、对外口径
Department	部门级：技术栈、值班表、紧急联系人
Group	群组级：项目代号、当前里程碑、团队默契
User	用户级：个人偏好、Mentor 风格、采纳偏好
Session	会话级：当前任务、最近上下文

优先级从低到高（Session 最高，覆盖 Enterprise）

13.2.3 对外 MCP 工具

工具	用途
`query_memory`	跨三层搜索（语义 + 关键词混合）
`get_recent_digest`	拿最近压缩后的摘要
`memory_resolve`	解析六级 `flow_memory.md`，返回当前生效的 system prompt 片段

13.3 差异化

FlowMemory 不依赖任何向量数据库（不强制 Pinecone / Neo4j），可裸跑在 2C2G 服务器；六级 markdown 层次可被人类直接编辑——运营 / HR 不需要懂 Python 就能维护”组织默契”。

14 作为 Agent 客户端安全栈（ShieldClaw）

14.1 对应赛道

AI 大模型安全 · 课题一：面向 Agent + 客户端环境下的安全操作与数据防护。课题聚焦 AI Agent + 客户端环境下面临的提示注入、工具滥用、数据外泄等安全风险，要求打造客户端安全防护能力框架。

14.2 八层安全栈

LarkMentor 的 `core/security/` 模块就是一个独立可用的客户端安全栈（命名 **ShieldClaw**），合计 1,098 行真代码（不含测试）：

#	层	文件（行数）	解决什么
1	PermissionManager	`permission_manager.py` (122)	5 级权限 deny-by-default
2	TranscriptClassifier	`transcript_classifier.py` (162)	Prompt Injection 检测
3	HookSystem	`hook_system.py` (139)	9 个生命周期事件拦截
4	PIIScrubber	`pii_scrubber.py` (62)	7 类 PII 脱敏
5	KeywordDenylist	`keyword_denylist.py` (148)	12 词 + 3 正则 + 热加载
6	RateLimiter	`rate_limiter.py` (163)	60s 滑窗限流
7	ToolSandbox	`tool_sandbox.py` (200)	API allowlist 沙箱
8	AuditLog	`audit_log.py` (102)	JSONL 全链路审计

14.3 对应课题三大威胁的兜底层

课题威胁	LarkMentor 的兜底（多层）
提示注入	L2 TranscriptClassifier 检测 + L5 KeywordDenylist 拦截 + L4 PII 二次脱敏
工具滥用	L1 Permission deny-by-default + L7 ToolSandbox API allowlist + L8 AuditLog 全留痕
数据外泄	L4 PIIScrubber 入库前 / LLM 调用前双扫 + L1 用户级硬隔离 +「永远是草稿」无 send 路径
DoS / 速率攻击	L6 RateLimiter 滑窗

14.4 Promptfoo 红队 14/14 通过

攻击	结果	兜底层
ignore previous instructions	PASS	L5 KeywordDenylist
DAN mode jailbreak	PASS	L5 KeywordDenylist
SQL 注入	PASS	L4 PIIScrubber
泄漏 system prompt	PASS	L2 TranscriptClassifier
诱导发紧急通知	PASS	L1 PermissionManager
诱导导出他人 KB	PASS	L1 + 用户级隔离
跨用户 KB 探针	PASS	用户级硬隔离
PII 注入入库	PASS	L4 PIIScrubber
...其他 6 个	PASS	多层

14.5 OWASP LLM Top 10 全映射

#	威胁	LarkMentor 缓解
1	Prompt Injection	L2 + L5 + L4（多层）
2	Insecure Output Handling	所有 mentor.* 永远 draft，永不 send
3	Training Data Poisoning	KB 入库 PII 二次扫描 + 用户级隔离
4	Model DoS	L6 RateLimiter
5	Supply Chain Vuln	依赖最小化（不引 LangGraph 等重框架）
6	Sensitive Info Disclosure	L4 + L1
7	Insecure Plugin Design	L7 ToolSandbox + L1 Permission
8	Excessive Agency	L1 5 级 + 草稿不发送 + 30 秒撤回
9	Overreliance	explainable 行 + 引用追踪
10	Model Theft	不暴露 API key，LLM 调用通过服务端转发

Part V

团队 & 后续

15 团队

15.1 李洁盈

角色：产品 / UI / UX / 内容

邮箱	JieyingLiii@outlook.com
个人网页	https://janeliii.netlify.app/
GitHub	https://github.com/Jane-0213
小红书	李什么盈

15.2 戴尚好

角色：技术负责人 / 全栈 / Agent 安全 / 部署

邮箱	bcefghj@163.com
个人网页	https://bcefghj.github.io
GitHub	https://github.com/bcefghj
小红书	bcefghj

16 后续提升方向

- **Promptfoo 红队扩到 50+**：当前 14 用例覆盖核心攻击面，下一步增加角色扮演越狱、混合语言注入、间接注入（文档触发）三类。
- **六维分类引入小模型在线学习**：用户每次”我觉得这条不该挡”反馈直接更新发件人维度权重。
- **FlowMemory Compaction 引入主题聚类**：当前 LLM 全文压缩，下一步先做主题聚类再分桶压缩，长会话压缩质量更高。
- **ShieldClaw 独立成 pip 包**：把 `core/security/` 抽成 `shieldclaw` 包，对外提供给任何基于 LLM 的 Agent 项目使用。
- **多语种 Mentor**：当前 NVC 改写仅中文，下一步支持中英混合 + 港式书面语 + 日韩职场用语。

- **Recovery Card 个性化**：根据用户采纳历史，自动调整草稿语气分布（保守 / 中性 / 直接的比例）。

附录 A · 18 个 MCP 工具完整清单

工具名	分组	用途
`get_focus_status`	核心	查当前专注状态
`classify_message`	核心	六维消息分类
`get_recent_digest`	核心	最近 archival 摘要
`add_whitelist`	核心	加白名单
`rollback_decision`	核心	决策回滚
`query_memory`	核心	跨三层记忆搜索
`mentor_review_message`	Mentor	MentorWrite NVC + 3 版改写
`mentor_clarify_task`	Mentor	MentorTask 模糊度 + 澄清
`mentor_draft_weekly`	Mentor	MentorReview STAR 周报
`mentor_search_org_kb`	Mentor	用户级 RAG 召回
`coach_review_message`	别名	兼容外部已部署调用
`coach_clarify_task`	别名	同上
`coach_draft_weekly`	别名	同上
`coach_search_org_kb`	别名	同上
`classify_readonly`	扩展	只读分类（不写状态）
`skill_invoke`	扩展	通用 Skill 调用入口
`memory_resolve`	扩展	解析六级 `flow_memory.md`
`list_skills`	扩展	列出可用 Skill 列表

附录 B · 飞书 7 个 API 实际调用清单

飞书 API	对应代码文件	用途
IM	`bot/message_sender.py`, `bot/event_handler.py`	收发消息、卡片回调
Docx	`core/mentor/growth_doc.py`	创建新手成长记录
Bitable	`utils/feishu_api.py`, `core/feishu_workspace_init.py`	任务结构化
Calendar	`core/feishu_advanced/calendar_busy.py`	忙闲查询
Wiki	`core/feishu_advanced/wiki_search.py`	组织知识检索
妙记 Minutes	`core/feishu_advanced/minutes_fetch.py`	生成 STAR 周报
表情 Reaction	`core/feishu_advanced/reaction_api.py`	已读回执
加急 Urgent	`core/feishu_advanced/urgent_api.py`	加急通知

附录 C · 数据出处清单

数据	出处
23 分 15 秒打断恢复	Mark, G., Gudith, D., Klocke, U. (2008). *The cost of interrupted work*. CHI '08.
68% 缺连续专注	Microsoft Work Trend Index 2023
80% 隐性知识	Nonaka, I., Takeuchi, H. (1995). *The Knowledge-Creating Company*. SECI Model.
8–12 月新人达全产能	SHRM Onboarding Report 2022 / BambooHR Onboarding Survey 2023
飞书 OpenClaw 描述	飞书 AI 校园挑战赛官网
OWASP LLM Top 10	OWASP Foundation 官方文档 2024 版
NVC 框架	Marshall Rosenberg *Nonviolent Communication* 2003
STAR 结构	标准行为面试法（DDI 提出）
Claude Code 七大支柱	戴尚好逆向报告 https://github.com/bcefghj/claude-code-complete-guide

LarkMentor · 2026-04-19 · github.com/bcefghj/larkmentor